



Solution Guide: Variables and If Statements

In this activity, you worked with `if` statements and variables, implementing them into your script if possible.

Using these tools will improve your script, making it more functional and logical.

Solutions

Get started by logging into the lab environment with the username `sysadmin` and password `cybersecurity`.

- Open the `sys_info.sh` script from the previous class using `nano`.
- Run `nano sys_info.sh`.

Using Variables

1. Create a variable to hold the path of your output file.

- Replace the output file path for each command with your variable.

Solution:

- `output=$HOME/research/sys_info.txt`
- We can now refer to this variable throughout the script. Instead of using `> /research/sys_info.txt`, we'll use `> $output`.

Break down the syntax:

- `output=`: This is the variable assignment. Remind students that there can be no spaces on either side of the `=` in Bash.
- `$HOME`: This is a built-in variable that is equal to `~` or the home folder path of the current user.
- `/research/sys_info.txt`: This is the path to our output file.

Now, we'll replace the output file path for each command with `>> $output`:

Unset

- `echo "A Quick System Audit Script" >> $output`
- `date >> $output`
- `echo "" >> $output`
- `echo "Machine Type Info:" >> $output`
- `echo -e "$MACHTYPE \n" >> $output`
- `echo -e "Uname info: $(uname -a) \n" >> $output`
- `echo -e "IP Info:" >> $output`
- `...`

Using If Statements

1. Create an if statement that checks for the existence of the `~/research` directory. If the directory exists, do nothing. If the directory does not exist, create it.

- Remove the line in your script that creates this directory.

Solution:

- First, remove:
`mkdir ~/research 2> /dev/null`
- Then, replace it with an if statement that checks for the existence of the `~/research` directory.

Unset

- `if [ ! -d $HOME/research ]`
- `then`
- `mkdir $HOME/research`
- `fi`

Syntax breakdown:

- `if` initiates the `if` statement.
- `[]` square brackets surround our conditional statement.
- `!` reverses the conditional statement that follows. (If this directory does *not* exist...)
- `-d` checks for the existence of the following directory.

- `$HOME/research` is our `$HOME` variable with the `research` directory appended.

It comes together as `if [ ! -d $HOME/research ]: "IF, NOT, Directory, ~/research" or "If the directory ~/research does not exist."`

- `then:` if the condition is met, run the following command.
- `mkdir $HOME/research` is the command run if `[ ! -d $HOME/research ]` is true.
- `fi` to close out our if statement.

Note that we only do an action, i.e., create the directory, if it does not already exist. If it does already exist, we do nothing.

We could add an `else` clause that tells the user that the directory already exists, but it's not necessary.

Bonus Variables

1. Create a variable to hold the output of the command `ip addr | grep inet | tail -2 | head -1`.
- Replace this command in your script with your new variable.

Solution:

- `ip=$(ip addr | head -9 | tail -1)`.
- Now, when the script runs, we have the IP info stored into a variable `ip`. We can call this variable with `$ip` and print its contents with `echo $ip`.

Syntax breakdown:

- `ip=` is our variable assignment.
- `$()` is our expansion syntax that tells Bash to "run this command first."
- `ip addr | head -9 | tail -1` is our compound command from the last class that gives us the IP address.

Now, we'll find the line in the script where this command runs and replace it with `echo $ip`:

Unset

- `echo -e "IP Info:" >> $output`
- `echo -e "$ip \n" >> $output`

Compare the above to what the code was previously. Note how much more streamlined the new code is.

Unset

- `echo "IP Info: $(ip addr | head -9 | tail -1) \n" >> ~/research/sys_info.txt`

2. Create a variable to hold the output of the command `find /home -type f -perm 777`.
 - Replace this command in your script with your new variable.

Solution:

- `execs=$(find /home -type f -perm 777)`
- This gives us the list of `exec` files in a variable `execs`.
- We can call it using `$execs` and print its contents using `echo $execs`.

Now, we'll replace the `find` command in the script with the new syntax:

Unset

- `echo -e "\nexec Files:" >> $output`
- `echo $execs >> $output`

Note that we only need to use the `-e` flag for `echo` if we want to use `\n` to create a new line.

Bonus If Statement

1. Create an if statement that checks for the existence of the file `~/research/sys_info.txt`.
 - If the file does not exist, do nothing.
 - If the file does exist, remove it. (This will ensure that the script always creates a new file.)

Solution:

Unset

- `if [ -f $output ]`
- `then`
- `rm $output`
- `fi`

Syntax breakdown:

- `if [ -f $output ]:` "If the file `$output` exists"

- `then rm $output: "then remove the output file"`
 - `fi` ends the `if` statement.
-

Bonus:

- Create an `if` statement that checks if the script was run using `sudo`.
- If it was run with `sudo`, do nothing.
- If it was run with `sudo`, exit the script with a message that tells the user not to run the script using `sudo`.

Solution:

First, we'll create an `if` statement that checks to see if the script was run using `sudo`.

Unset

```
○ if [ $UID -ne 0 ]
○ then
○   echo "Please run this script with sudo."
○   exit
○ fi
```

Syntax breakdown:

- `if [ $UID -ne 0 ]:` "If \$UID does not equal zero..."
- `$UID` variable will print the UID of the user. The root user is always 0, making this an easy conditional to check.
- `then echo "Please run this script with sudo."` ...then print a message to the user.
- `exit:` Stops the script.
- `fi:` Ends the `if` statement.

Note that we do not need to specify that nothing will happen if the user is not `root`.

There are a number of ways to write this statement. Provide a few other examples:

- `if [ $USER = 'root' ]` will check the contents of the `$USER` variable against 'root.'
- `if [ $(whoami) = 'root' ]` will check the output of the `whoami` command against 'root.'

At this point, your script should look similar to this:

Unset

```
○ #!/bin/Bash
○
○ #Check if script was run as root. Exit if false.
○ if [ $UID -ne 0 ]
○ then
○     echo "Please run this script with sudo."
○     exit
○ fi
○
○ # Define Variables
○ output=$HOME/research/sys_info.txt
○ ip=$(ip addr | grep inet | tail -2 | head -1)
○ execs=$(find /home -type f -perm 777 2> /dev/null)
○
○ # Check for research directory. Create it if needed.
○ if [ ! -d $HOME/research ]
○ then
○     mkdir $HOME/research
○ fi
○
○ # Check for output file. Clear it if needed.
○ if [ -f $output ]
○ then
○     rm $output
○ fi
○
○ echo "A Quick System Audit Script" >> $output
○ date >> $output
○ echo "" >> $output
○ echo "Machine Type Info:" >> $output
○ echo -e "$MACHTYPE \n" >> $output
○ echo -e "Uname info: $(uname -a) \n" >> $output
○ echo -e "IP Info:" >> $output
○ echo -e "$ip \n" >> $output
○ echo -e "Hostname: $(hostname -s) \n" >> $output
○ echo "DNS Servers: " >> $output
```

- `cat /etc/resolv.conf >> $output`
 - `echo -e "\nMemory Info:" >> $output`
 - `free >> $output`
 - `echo -e "\nCPU Info:" >> $output`
 - `lscpu | grep CPU >> $output`
 - `echo -e "\nDisk Usage:" >> $output`
 - `df -H | head -2 >> $output`
 - `echo -e "\nWho is logged in: \n $(who -a) \n" >> $output`
 - `echo -e "\nexec Files:" >> $output`
 - `echo $execs >> $output`
 - `echo -e "\nTop 10 Processes" >> $output`
 - `ps aux --sort -%mem | awk {'print $1, $2, $3, $4, $11'} | head >> $output`
 - `fi`
-